

Bartłomiej Gorski, Michał Sergiel, Szymon Dłubała, Tomasz Stępczyński, Zbigniew Sztanga

Politechnika Warszawska

Projekt PAINT

Sklep wędkarski – Dokumentacja końcowa

7 czerwca 2026

Spis treści

1. Temat projektu i założenia funkcjonalne	2
2. Opis członków i ról w zespole	2
3. Architektura projektu	3
3.1. Warstwa prezentacji	3
3.2. Warstwa aplikacji	3
3.3. Warstwa danych	4
3.4. Komunikacja i wdrożenie	4
4. Wykorzystane narzędzia i technologie	4
4.1. Spring Boot	5
4.2. React	5
4.3. Tailwind CSS	5
4.4. Docker + Docker Compose	5
4.5. Nginx	5
4.6. Tailscale Funnel	5
5. Opis realizacji projektu	6
6. Opis techniczny produktu	6
6.1. Moduły warstwy prezentacji (UX/UI)	6
7. Instrukcja instalacji	7
7.1. Wymagania	7
7.2. Instalacja od czystego systemu	7
8. Instrukcja użytkownika	8
8.1. Strona główna	8
8.2. Ekran logowania	8
8.3. Sklep	9
8.4. Rezerwacja łowiska	9
8.5. Rekomendacja sprzętu	10
8.6. Koszyk	10
8.7. Wybór metody płatności	11
8.8. Panel administratora	11



**Wydział Elektroniki
i Technik Informatycznych**

1. Temat projektu i założenia funkcjonalne

Tematem projektu jest aplikacja webowa dla sklepu wędkarskiego połączonego z systemem rezerwacji stanowisk na łowiskach. Produkt ma umożliwiać klientowi przeglądanie oferty sprzętu, dobór produktów do wybranego łowiska, rezerwację stanowiska w określonym terminie oraz złożenie zamówienia obejmującego zarówno produkty, jak i usługę rezerwacji.

Projekt został zrealizowany jako aplikacja full-stack:

- dynamiczny frontend WWW zbudowany w React,
- backend zrealizowany w Spring Boot,
- relacyjna baza danych MySQL,
- środowisko uruchomieniowe oparte o Docker Compose, Nginx i kontenery dla warstwy frontendowej, backendowej oraz bazy danych.

Najważniejsze założenia funkcjonalne produktu:

- **Katalog produktów** – wyświetlanie asortymentu sklepu, wyszukiwanie tekstowe, filtrowanie po kategoriach, filtrowanie po maksymalnej cenie, prezentacja ceny i stanu magazynowego.
- **Szczegóły produktu** – osobny widok produktu z opisem, kategorią, ceną, stanem magazynowym i możliwością dodania do koszyka.
- **Interaktywny plan łowiska** – lista łowisk pobierana z bazy danych, prezentacja zdjęcia wybranego łowiska, pinezki stanowisk oraz wybór daty i przedziału czasowego rezerwacji.
- **Rezerwacja stanowiska** – dodanie rezerwacji do koszyka, kontrola zajętości na podstawie istniejących zamówień oraz rozróżnienie stanowisk dostępnych i wyłączonych z rezerwacji.
- **Rekomendacje produktowe** – po dodaniu rezerwacji aplikacja pobiera produkty powiązane z rybami występującymi na wybranym łowisku i prezentuje rekomendacje.
- **Koszyk** – przechowywanie produktów oraz rezerwacji stanowiska, usuwanie pojedynczych produktów, anulowanie rezerwacji, wyliczenie łącznej wartości koszyka oraz przejście przez krok płatności.
- **Zamówienia** – utworzenie zamówienia, zapis pozycji zamówienia oraz wyczyszczenie koszyka po udanym złożeniu zamówienia.
- **Konto użytkownika** – rejestracja, logowanie, edycja danych profilu, wylogowanie, podgląd aktywnych rezerwacji i historii zamówień.
- **Panel administracyjny** – kreator łowisk pozwalający wizualnie dodać zdjęcie, oznaczyć stanowiska oraz dodać łowisko do bazy.

2. Opis członków i ról w zespole

- **Szymon Dłubała - Baza danych**
 - Zaprojektowanie diagramu relacyjnego systemu sklepowego.
 - Implementacja modelu fizycznego bazy.
 - Opracowanie mechanizmów zapewniających integralność danych.
 - Optymalizacja typów danych w celu usprawnienia działania oraz ograniczenia rozmiaru modelu.
 - Inicjalizacja systemu przykładowymi rekordami.
- **Bartłomiej Gorski - Konteneryzacja i Hosting**
 - Przygotowanie konfiguracji kontenerów dla wszystkich części aplikacji.
 - Przygotowanie infrastruktury sieciowo-serwerowej.
 - Opracowanie instrukcji instalacji i uruchomienia aplikacji.
- **Tomasz Stępczyński - Integracja z bazą danych**
 - Konfiguracja połączenia aplikacji z bazą danych.
 - Implementacja repozytoriów oraz serwisów obsługujących logikę biznesową.
 - Przygotowanie modeli danych odzwierciedlających strukturę bazy w kodzie Javy.

- **Michał Sergiel - UX & UI**
 - Opracowanie głównego szkieletu aplikacji (Layout, Navigation Bar, Footer).
 - Implementacja modułu interaktywnej mapy łowiska.
 - Przygotowanie widoków panelu użytkownika oraz statycznych szablonów dokumentacji profilu.
 - Nadzór nad spójnością wizualną projektu.
- **Zbigniew Sztanga - UX & UI**
 - Implementacja logiki katalogu produktów (wyświetlanie, sortowanie).
 - Opracowanie mechanizmu koszyka zakupowego oraz stanów dynamicznych aplikacji.
 - Integracja warstwy frontendowej z usługami API oraz obsługa odpowiedzi z serwera.
 - Walidacja formularzy i obsługa komunikatów błędów pochodzących z backendu.

3. Architektura projektu

Aplikacja została zaprojektowana jako system trójwarstwowy. Warstwa prezentacji komunikuje się z backendem przez REST API, backend realizuje logikę biznesową i korzysta z repozytoriów JPA, a dane są przechowywane w relacyjnej bazie MySQL.

3.1. Warstwa prezentacji

Warstwa prezentacji została zrealizowana jako aplikacja typu Single Page Application (SPA) przy użyciu biblioteki React. Architektura opiera się na komponentach, co umożliwia ponowne wykorzystanie kodu oraz łatwe zarządzanie stanem wizualnym interfejsu. Za nawigację między widokami odpowiada biblioteka React Router, co pozwala na płynne przełączanie stron bez przeładowywania całej aplikacji. Zastosowano responsywny układ, gwarantujący poprawne wyświetlanie zarówno na urządzeniach mobilnych, jak i ekranach komputerów. Szkielet frontendu został zaprojektowany z myślą o pełnej integracji z usługami API, eksponując jasne interfejsy dla dynamicznych danych ładowanych z backendu.

3.2. Warstwa aplikacji

Warstwa aplikacji została zrealizowana w oparciu o framework Spring Boot, który zapewnia tworzenie aplikacji backendowych opartych na architekturze REST. Komunikacja między warstwą prezentacji a warstwą aplikacji odbywa się wyłącznie poprzez interfejs REST API, w którym dane wymieniane są w formacie JSON. Każdy kontroler udostępnia zasoby pod określonym adresem URL, obsługując żądania HTTP metodami GET, POST, PUT oraz DELETE.

W systemie wyróżniono następujące moduły funkcjonalne:

- **Moduł użytkowników** – obsługiwany przez `UserController` oraz `UserService`, realizuje rejestrację i logowanie użytkowników, pobieranie oraz aktualizację danych profilu, a także operacje takie jak przeglądanie listy wszystkich użytkowników i ich usuwanie.
- **Moduł produktów** – obsługiwany przez `ProductController` oraz `ProductService`, udostępnia pełen zestaw operacji CRUD na produktach sklepu. Umożliwia pobieranie wszystkich produktów, filtrowanie po kategorii, wyszukiwanie po identyfikatorze, a także dodawanie, edycję i usuwanie produktów przez administratora. Dodatkowo zaimplementowano wyszukiwanie produktów rekomendowanych dla danego łowiska – endpoint `GET /api/produkty/lowisko/{fisheryId}` zwraca produkty powiązane z rybami występującymi w wybranym łowisku, wykorzystując relacje Many-to-Many między encjami `Product`, `Fish` oraz `Fishery`.
- **Moduł łowisk** – obsługiwany przez `FisheryController` oraz `FisheryService`, umożliwia pobieranie listy wszystkich łowisk, szczegółów wybranego łowiska oraz dodawanie nowych łowisk. Kluczową funkcjonalnością jest wyszukiwanie łowisk po gatunku ryby – endpoint `GET /api/fisheries/ryba/{fishName}` zwraca łowiska, w których występuje ryba o podanej nazwie.
- **Moduł stanowisk wędkarskich** – obsługiwany przez `FishingStandController` oraz `FishingStandService`, realizuje pobieranie stanowisk przypisanych do konkretnego łowiska, aktualizację dostępności stanowiska oraz dodawanie nowych stanowisk. Każde stanowisko przechowuje współrzędne `x_pos` i `y_pos`, które umożliwiają frontendowi renderowanie jego położenia na mapie łowiska.

- **Moduł koszyka** – obsługiwany przez `CartController`, umożliwia zarządzanie zawartością koszyka zalogowanego użytkownika: dodawanie produktów z validacją stanu magazynowego, usuwanie pozycji, czyszczenie koszyka oraz zarządzanie rezerwacją stanowiska wędkarskiego powiązaną z koszykiem.
- **Moduł zamówień** – obsługiwany przez `OrderController` oraz `OrderService`, obsługuje tworzenie zamówień i pozycji zamówień, pobieranie historii zamówień użytkownika, a także operacje: przeglądanie wszystkich zamówień, zmianę statusu zamówienia oraz jego usuwanie.

3.3. Warstwa danych

Warstwa danych stanowi fundament systemu, odpowiadając za trwałe przechowywanie informacji o asortymencie sklepu, użytkownikach, procesach zakupowych oraz infrastrukturze łowisk. Została ona zaimplementowana w oparciu o relacyjny model danych ACID, co zapewnia spójność oraz integralność informacji poprzez system kluczy obcych i więzów spójności.

W skład tej warstwy wchodzi następujące komponenty:

- **Relacyjna baza danych:** Przechowuje dane w ustrukturyzowanych tabelach. Kluczowe encje to: użytkownik, produkt, ryba oraz łowisko wraz z przypisanymi do niego stanowiskami.
- **Logika powiązań Many-to-Many:** Zastosowano tabele łączące, takie jak `produkt_ryba` oraz `lowisko_ryba`. Pozwala to na definiowanie fauny występującej na danym akwenie oraz rekomendacja odpowiednich produktów.
- **System współrzędnych stanowisk:** Tabela `stanowisko` przechowuje dane o lokalizacji punktów wędkarskich za pomocą kolumn `x_pos` i `y_pos` typu `DECIMAL(5,2)`. Wartości te są wyrażone w procentach, co umożliwia responsywne skalowanie punktów na mapach łowisk niezależnie od rozdzielczości urządzenia końcowego.
- **Zasoby binarne:** Ścieżki do plików multimedialnych są przechowywane w bazie w formie relatywnych ścieżek tekstowych (np. `/images/products/rods/madcat_white_deluxe.jpg`), co pozwala na separację bazy od fizycznych plików graficznych serwowanych przez warstwę prezentacji.
- **Automatyzacja zasileń:** Mechanizm inicjalizacji bazy oparty jest na skryptach: `01-init.sql` odpowiadającego za strukturę oraz `02-data.sql` odpowiadającego za inicjalizację przykładowymi danymi. Dzięki temu system po uruchomieniu w kontenerze Docker posiada pełny zestaw danych demonstracyjnych, w tym zdefiniowane zamówienia i koszyki użytkowników.

3.4. Komunikacja i wdrożenie

W środowisku kontenerowym Nginx jest punktem wejścia do aplikacji. Udostępnia zbudowany frontend React na porcie 80, a zapytania zaczynające się od `/api/` przekazuje do kontenera backendu Spring Boot. Dzięki temu frontend może wykonywać żądania pod względnymi adresami, np. `/api/produkty`, bez wpisywania osobnego adresu backendu.

Backend komunikuje się z bazą danych MySQL wewnątrz sieci Docker Compose przez adres usługi `mysql:3306`. Sama baza została dodatkowo wystawiona na komputer gospodarza jako `localhost:3307`, aby możliwe było połączenie się z nią z zewnętrznych narzędzi, np. DataGrip. Mapowanie portów ma więc postać `3307:3306`: port 3306 działa wewnątrz kontenera MySQL, a port 3307 jest dostępny lokalnie na gościu.

4. Wykorzystane narzędzia i technologie

MySQL 8.4

Jako system zarządzania relacyjną bazą danych (RDBMS) wybrano MySQL w wersji 8.4. Jest to jedna z najpopularniejszych otwartoźródłowych baz danych, charakteryzująca się wysoką wydajnością i niezawodnością. Wybór tej technologii podyktowany był następującymi czynnikami:

- **Zgodność ze standardami:** Pełna obsługa standardu SQL umożliwia tworzenie złożonych zapytań łączących wiele tabel, co umożliwia proste generowanie raportów zamówień i zaawansowane filtrowanie produktów.
- **Typy danych:** Wykorzystanie precyzyjnych typów numerycznych (np. `DECIMAL` dla cen i koordynatów) eliminuje błędy zaokrągleń, które mogłyby wystąpić przy typach zmiennoprzecinkowych.
- **Konteneryzacja:** Obraz MySQL 8.4 jest w pełni kompatybilny z technologią Docker, co pozwala na błyskawiczne postawienie całego środowiska bazodanowego bez konieczności lokalnej konfiguracji serwera na maszynie prowadzącego.
- **Wydajność:** Nowoczesny silnik InnoDB zapewnia wsparcie dla transakcji oraz blokowanie na poziomie wiersza, co gwarantuje bezpieczeństwo danych przy jednoczesnych operacjach na koszykach przez wielu użytkowników.

4.1. Spring Boot

Spring Boot został wybrany jako główny framework backendowy. Framework dostarcza wbudowany serwer Tomcat, co upraszcza wdrożenie aplikacji w środowisku kontenerowym.

Do mapowania obiektowo-relacyjnego wykorzystano bibliotekę Spring Data JPA z implementacją Hibernate. Repozytoria dziedziczą po `JpaRepository` lub `CrudRepository`, co zapewnia gotowe metody dostępu do danych bez konieczności pisania zapytań SQL. Tam, gdzie wymagane były niestandardowe zapytania, zastosowano adnotację `@Query` z językiem JPQL lub natywne zapytania SQL.

Bezpieczeństwo aplikacji zostało skonfigurowane przy użyciu Spring Security. Klasa `SecurityConfig` definiuje reguły dostępu do poszczególnych endpointów – część zasobów jest publiczna, natomiast operacje administracyjne wymagają odpowiedniej roli. Hasła użytkowników są szyfrowane z użyciem `BCryptPasswordEncoder`.

Konfiguracja aplikacji przechowywana jest w pliku `application.properties`, który definiuje parametry połączenia z bazą danych, kodowanie znaków oraz zachowanie Hibernate.

4.2. React

Główna biblioteka wykorzystana do budowy interfejsu użytkownika. Umożliwiła stworzenie modularnej struktury komponentów (takich jak główny szkielet aplikacji, pasek nawigacji, stopka, interaktywna mapa łowiska czy statyczne widoki profilu klienta) oraz wstępne zarządzanie stanem aplikacji na poziomie prezentacji.

4.3. Tailwind CSS

Narzędzie (framework) wykorzystane do stylowania warstwy wizualnej aplikacji. Tailwind CSS pozwolił na szybkie i spójne zaprojektowanie nowoczesnego, responsywnego interfejsu (UI) bez konieczności pisania rozbudowanych, zewnętrznych arkuszy stylów. Odpowiada za jednolity wygląd przycisków, układów stron oraz animacje widoczne m.in. podczas interakcji z mapą łowiska i okienkami.

4.4. Docker + Docker Compose

Docker umożliwia uruchomienie całego produktu w powtarzalnym środowisku. Plik `compose.yaml` definiuje trzy usługi:

- `mysql` – baza MySQL 8.4 z wolumenem danych i skryptami inicjalizacyjnymi,
- `backend` – obraz budowany z `Dockerfile.backend`,
- `web` – obraz budowany z `Dockerfile.web`, zawierający statyczny build React i Nginx.

4.5. Nginx

W środowisku kontenerowym Nginx udostępnia zbudowaną aplikację React jako pliki statyczne oraz pełni rolę pośrednika dla zapytań do backendu. Żądania zaczynające się od `/api/` są przekazywane do backendu, natomiast pozostałe ścieżki są kierowane do `index.html`, aby routing po stronie React'a działał również po odświeżeniu strony lub bezpośrednim wejściu na konkretny adres.

4.6. Tailscale Funnel

Tailscale Funnel wykorzystano do udostępnienia aplikacji poza lokalną siecią, bez konieczności ręcznej konfiguracji publicznego serwera DNS, przekierowania portów na routerze ani wykupienia osobnego hostingu. Dzięki temu aplikacja może być uruchomiona lokalnie na komputerze lub serwerze zespołu, a jednocześnie dostępna z Internetu pod publicznym adresem.

W ramach projektu aplikacja została udostępniona pod adresem:

<https://bartomix.tailc381e2.ts.net>

Backend został skonfigurowany tak, aby akceptował zapytania z adresów używanych przez frontend, w tym z domeny Tailscale. Pozwala to na poprawną komunikację przeglądarki z API również wtedy, gdy aplikacja jest prezentowana zdalnie.

5. Opis realizacji projektu

Prace nad projektem rozpoczęto od burzy mózgów, podczas której zespół analizował możliwe tematy aplikacji oraz zakres funkcjonalności możliwy do zrealizowania w ramach projektu. Po wyborze koncepcji sklepu wędkarskiego połączonego z systemem rezerwacji łowisk przystąpiono do definiowania głównych funkcji produktu. Określono, że aplikacja powinna obejmować katalog sprzętu wędkarskiego, koszyk zakupowy, konta użytkowników, interaktywną mapę łowisk oraz możliwość rezerwacji stanowisk.

Następnym etapem było przygotowanie makiety aplikacji i przedstawienie jej opiekunowi projektu. Po konsultacji zakres funkcjonalny został rozszerzony o dwa elementy: możliwość dodawania łowisk przez administratora oraz wsparcie użytkownika w doborze sprzętu wędkarskiego zależnie od wybranego łowiska. Druga funkcja została zrealizowana przez powiązanie łowisk, gatunków ryb i produktów, dzięki czemu aplikacja może rekomendować sprzęt pasujący do danego miejsca połowu.

Po zaakceptowaniu koncepcji rozpoczęto właściwą implementację. W pierwszej kolejności przygotowano początkowy model bazy danych, aby możliwe było rozpoczęcie prac nad backendem. Backend został zrealizowany w Spring Boot i obejmował encje JPA, repozytoria, serwisy oraz kontrolery REST odpowiedzialne za obsługę użytkowników, produktów, łowisk, stanowisk, koszyka i zamówień.

Równolegle powstawała warstwa frontendowa w React. Przygotowano główne widoki aplikacji: stronę główną, katalog produktów, szczegóły produktu, logowanie i rejestrację, profil użytkownika, koszyk, mapę rezerwacji oraz panel administracyjny. Po przygotowaniu pierwszych wersji frontendu i backendu połączono obie warstwy za pomocą REST API. Frontend zaczął pobierać dane z backendu i wysyłać żądania związane m.in. z logowaniem, dodawaniem produktów do koszyka, zapisem rezerwacji i tworzeniem zamówień.

Po integracji aplikację skonteneryzowano. Przygotowano osobne kontenery dla bazy danych MySQL, backendu Spring Boot oraz frontendu serwowanego przez Nginx. Dzięki Docker Compose możliwe stało się uruchomienie całego systemu jednym poleceniem oraz testowanie aplikacji w środowisku zbliżonym do docelowego.

Etap testów wykazał potrzebę dopracowania obsługi stanowisk na interaktywnej mapie. W szczególności konieczne było zapewnienie poprawnego rozmieszczenia pinezek reprezentujących stanowiska oraz spójnego przekazywania informacji o dostępności stanowisk między bazą danych, backendem i frontendem.

W związku z tym rozszerzono model stanowisk o dane wykorzystywane do pozycjonowania pinezek na mapie oraz dostosowano backend i frontend do obsługi tych informacji.

Finalnie projekt obejmuje działającą aplikację webową z katalogiem produktów, systemem kont użytkowników, koszykiem, składaniem zamówień, rezerwacją stanowisk, rekomendacją sprzętu na podstawie wybranego łowiska oraz panelem administratora umożliwiającym zarządzanie łowiskami i stanowiskami

6. Opis techniczny produktu

6.1. Moduły warstwy prezentacji (UX/UI)

W ramach budowy warstwy prezentacji opracowano szereg niezależnych komponentów, które wspólnie tworzą interfejs wizualny aplikacji:

- **Moduł nawigacji i układu (Layout, Navbar, Footer)** - Główne komponenty otaczające każdą podstronę, zapewniające globalną nawigację, dostęp do profilu oraz stopkę z informacjami.
- **Moduł interaktywnej mapy (MapArea)** - Komponent renderujący zdjęcie wybranego łowiska z nałożonymi pinezkami stanowisk. Wykorzystuje relatywne pozycjonowanie CSS (wartości procentowe) do umieszczania stanowisk na obrazie, co gwarantuje poprawność wyświetlania i klikalności niezależnie od rozdzielczości ekranu. Zawiera formularz wyboru daty i godzin rezerwacji oraz obsługuje wyświetlanie okna z rekomendacjami sprzętu.
- **Moduł panelu administracyjnego (AdminPanel)** - Wizualny kreator łowisk pozwalający na wgranie podglądu zdjęcia, precyzyjne nakładanie stanowisk wędkarskich poprzez kliknięcie na mapie oraz wizualne przypisywanie im rekomendowanego sprzętu z listy produktów.
- **Moduł koszyka zakupowego (Cart)** - Statyczny, wieloetapowy interfejs koszyka obejmujący listę wybranych produktów z opcją ich usunięcia, płynne przejście do formularza danych dostawy oraz sekcję wyboru metody płatności.
- **Moduł sklepu i detali produktu (Shop, ProductDetail)** - Szablony wizualne głównego katalogu produktów wędkarskich wraz z bocznym panelem filtrów oraz dedykowana strona szczegółów pojedynczego produktu.

- **Moduły konta użytkownika (Profile, EditProfile, Auth)** - Widoki odpowiadające za formularze logowania i rejestracji, wizualną prezentację danych użytkownika, historii zamówień oraz aktywnych rezerwacji w czytelnym układzie zakładowym.
- **Moduł obsługi błędów (ErrorPage)** - Uniwersalny komponent wizualny wyświetlany w przypadku wystąpienia błędów (np. 404, 500), przyjmujący kod błędu jako argument i dynamicznie dostosowujący wyświetlaną grafikę oraz komunikat.

7. Instrukcja instalacji

7.1. Wymagania

- Docker Engine lub Docker Desktop.
- Docker Compose w wersji drugiej, dostępny jako polecenie `docker compose`.
- Git, jeśli projekt ma zostać pobrany z repozytorium.
- Wolne porty 80 i 3307 na maszynie uruchomieniowej.

7.2. Instalacja od czystego systemu

1. Zainstalować Docker:
 - Linux: zainstalować pakiety Docker Engine zgodnie z instrukcją dystrybucji i dodać użytkownika do grupy `docker`, jeżeli jest to wymagane.
 - Windows/macOS: zainstalować Docker Desktop i uruchomić aplikację Docker.
2. Sprawdzić instalację:

```
docker --version
docker compose version
```

3. Pobrać projekt lub skopiować katalog projektu na maszynę docelową.
4. Przejść do katalogu głównego projektu, w którym znajdują się pliki `compose.yaml`, `Dockerfile.backend` i `Dockerfile.web`.
5. Uruchomić budowanie i start aplikacji:

```
docker compose up --build
```

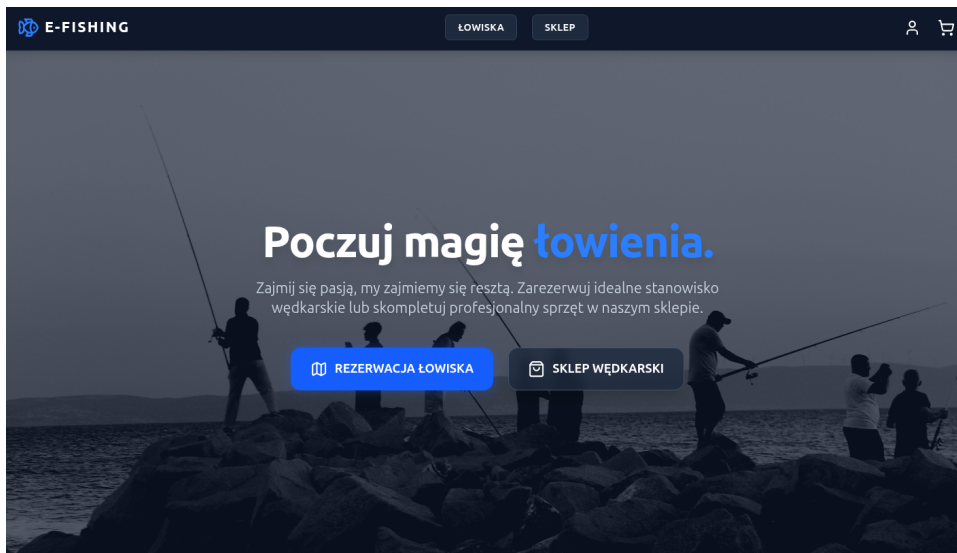
6. Począć, aż kontener `mysql` przejdzie `healthcheck`, backend uruchomi się na porcie 8080 wewnątrz sieci Compose, a kontener `web` zacznie nasłuchiwać na porcie 80.
7. Otworzyć aplikację w przeglądarce:

```
http://localhost/
```

Podczas pierwszego uruchomienia MySQL wykonuje skrypty z katalogu `init-db`. Dane bazy są przechowywane w wolumenie `mysql.prod.data`, dlatego kolejne uruchomienia zachowują stan bazy. Aby odtworzyć bazę od zera, należy usunąć wolumen danych i uruchomić Compose ponownie.

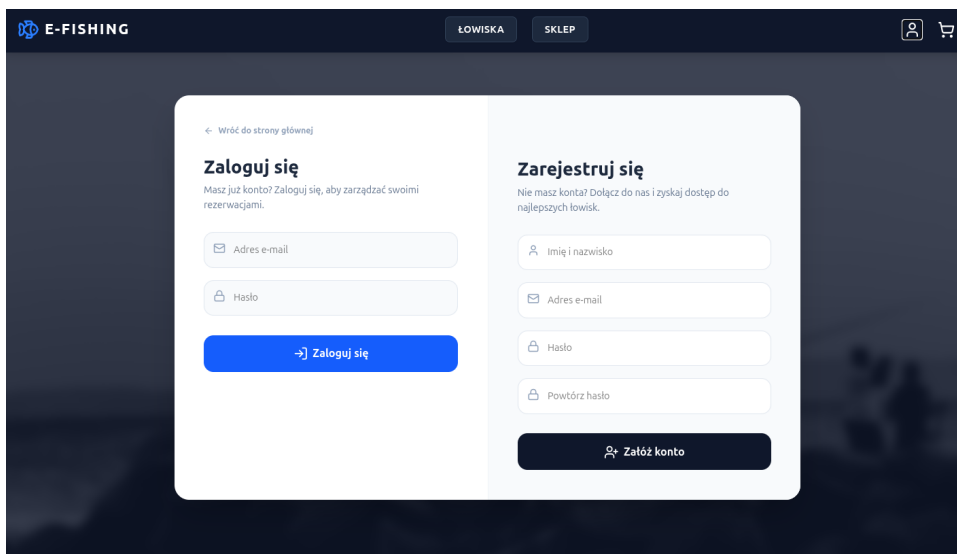
8. Instrukcja użytkownika

8.1. Strona główna



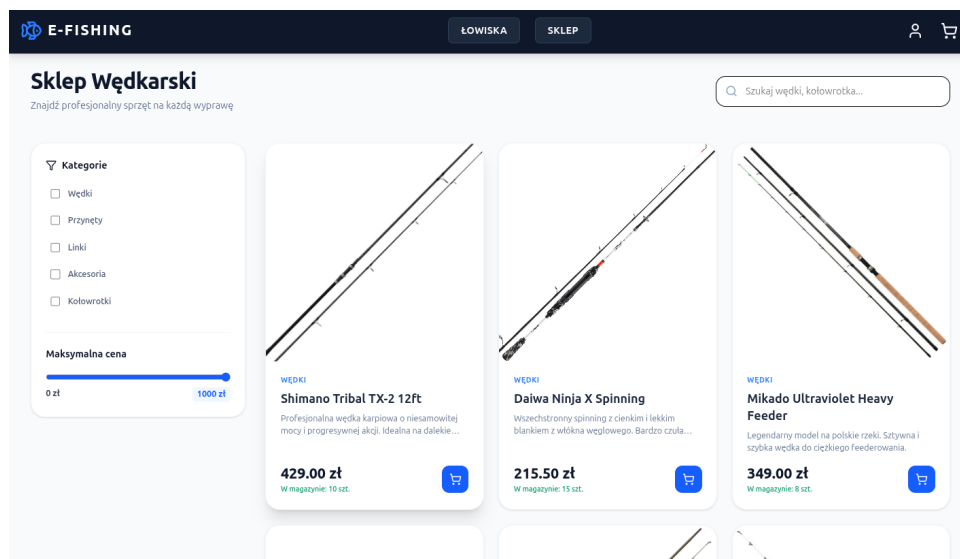
Strona główna umożliwia przejście do sklepu (na górnym pasku: "SKLEP" oraz w środkowej części "SKLEP WĘDKARSKI") oraz do systemu rezerwacji łowiska ("REZERWACJA ŁOWISKA" oraz "ŁOWISKA"). Aby w pełni wykorzystać funkcjonalność serwisu należy zalogować się klikając ikonę w kształcie sylwetki człowieka w prawym górnym rogu. Ikona koszyka obok pozwala na podgląd koszyka z produktami oraz rezerwacji (po zalogowaniu). Na ikonie koszyka wyświetlona jest również liczba produktów w koszyku. Dodatkowo pasek nawigacji widoczny na górze strony jest widoczny zawsze, i umożliwia powrót na stronę główną w każdym momencie poprzez kliknięcie w logo po lewej stronie.

8.2. Ekran logowania



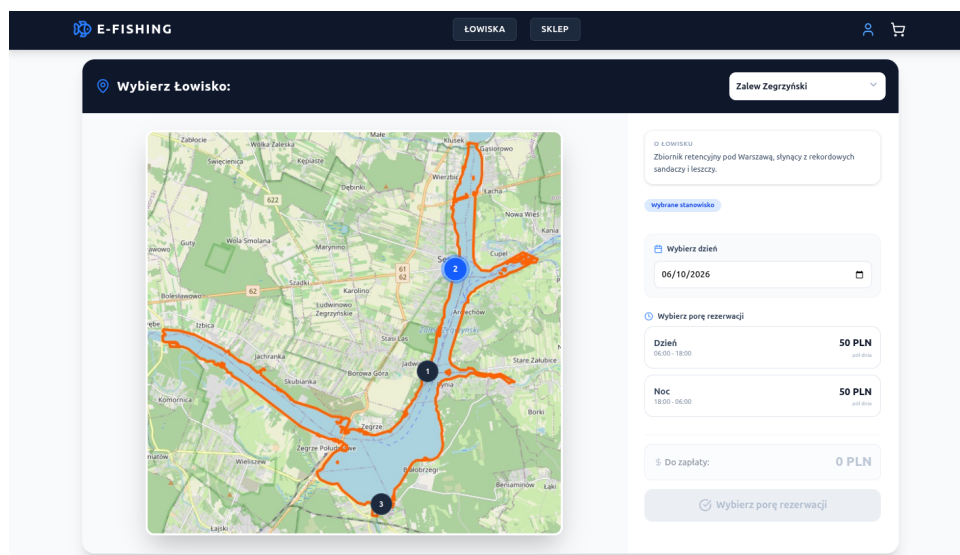
Ekran logowania i rejestracji pozwala na zalogowanie się na istniejące konto (formularz po lewej) oraz rejestrację nowego użytkownika (po prawej stronie). Przejście do tego ekranu odbywa się poprzez kliknięcie ikony "ludzika" w pasku nawigacji.

8.3. Sklep



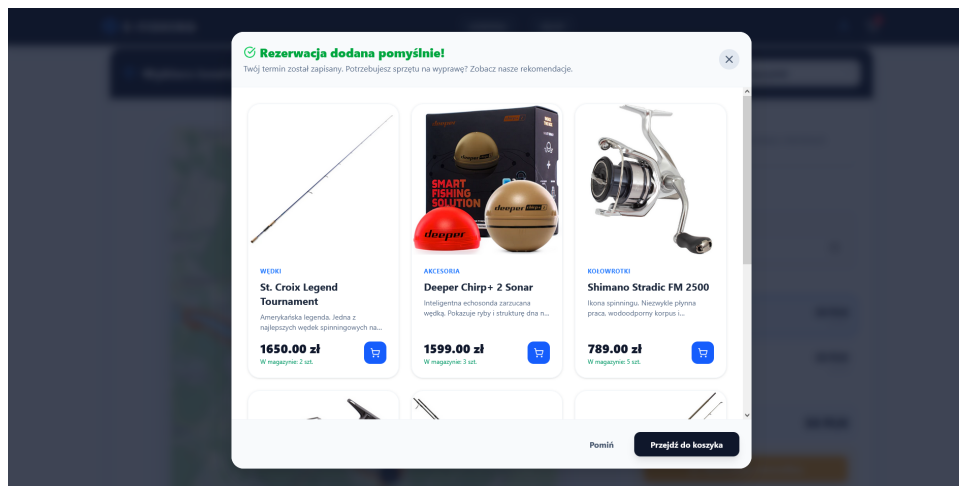
Strona sklepu umożliwia przeglądanie katalogu oraz filtrowanie produktów po kategoriach oraz cenie. Produkty można dodawać do koszyka poprzez niebieską ikonę widoczną w okienku każdego produktu. Dodatkowo po kliknięciu w produkt użytkownik zostaje odesłany do strony produktu z pełnym opisem. Tam również istnieje możliwość dodania produktu do koszyka.

8.4. Rezerwacja łowiska



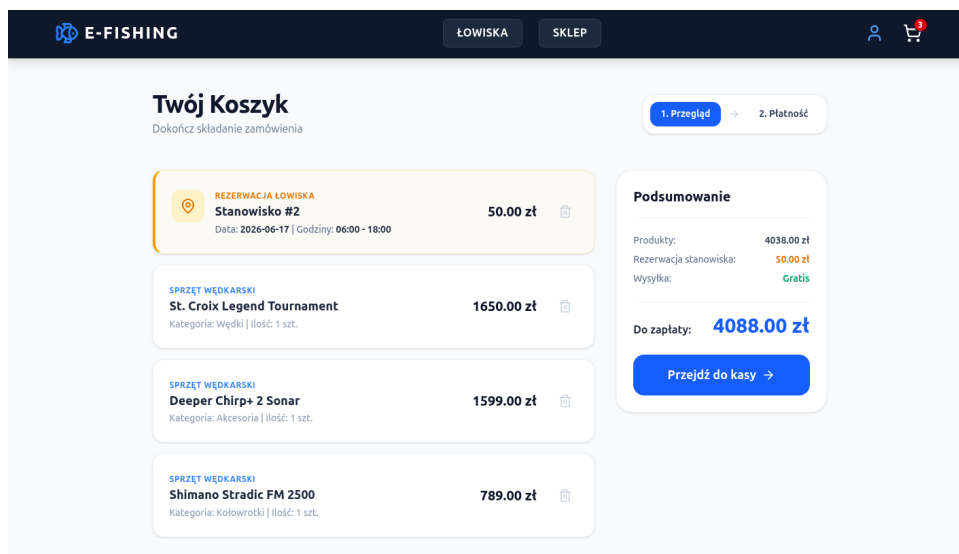
Rezerwacja łowiska możliwa jest poprzez dedykowany formularz. W prawym górnym rogu możliwy jest wybór jeziora. Po dokonaniu wyboru ukazuje się mapa wraz ze stanowiskami możliwymi do zarezerwowania. Jeśli dane stanowisko jest wyłączane pokazuje się na czerwono. Po kliknięciu stanowiska ukazuje się panel wyboru daty oraz czasu rezerwacji. Jeśli stanowisko jest zarezerwowane nie będzie możliwe wybranie danej daty i czasu. Zatwierdzenie możliwe jest przyciskiem na dole prawego panelu. Możliwe jest dokonanie jednej rezerwacji na zamówienie. Więcej rezerwacji nie jest możliwe, a użytkownik jest poinformowany odpowiednim komunikatem.

8.5. Rekomendacja sprzętu



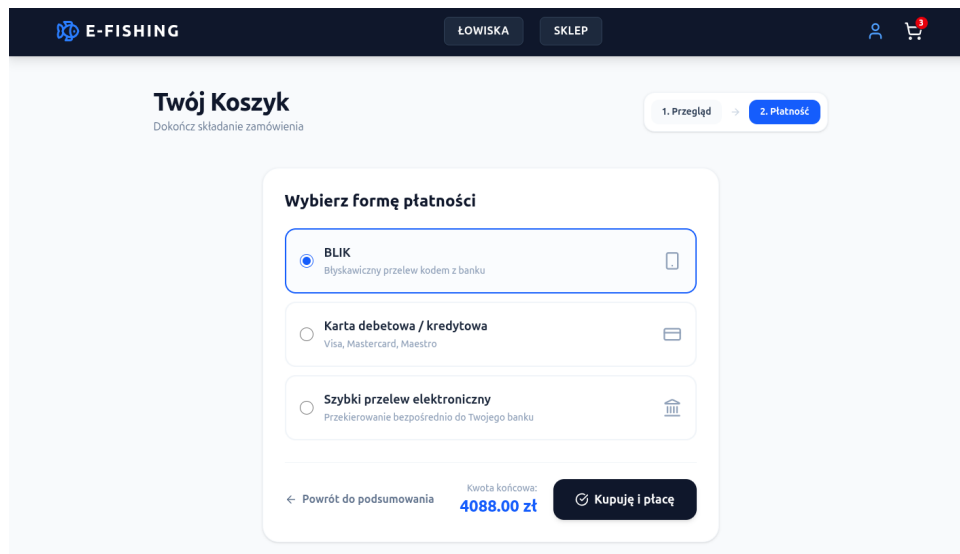
Po pomyślnej rezerwacji wyświetlane jest okno, w którym użytkownik dostaje 6 rekomendacji produktów na podstawie zarezerwowanego łowiska. Rekomendacje dobierane są na podstawie ryb dostępnych w danej lokalizacji. Z tego miejsca możliwe jest dodanie produktów do koszyka oraz wejście w stronę danego produktu. Zamknięcie okna możliwe jest przyciskiem "x", pomiń oraz opcją "przejdź do koszyka". Pierwsze dwie opcje nie przekierowują użytkownika dalej. Opcja trzecia odsyła użytkownika do koszyka.

8.6. Koszyk



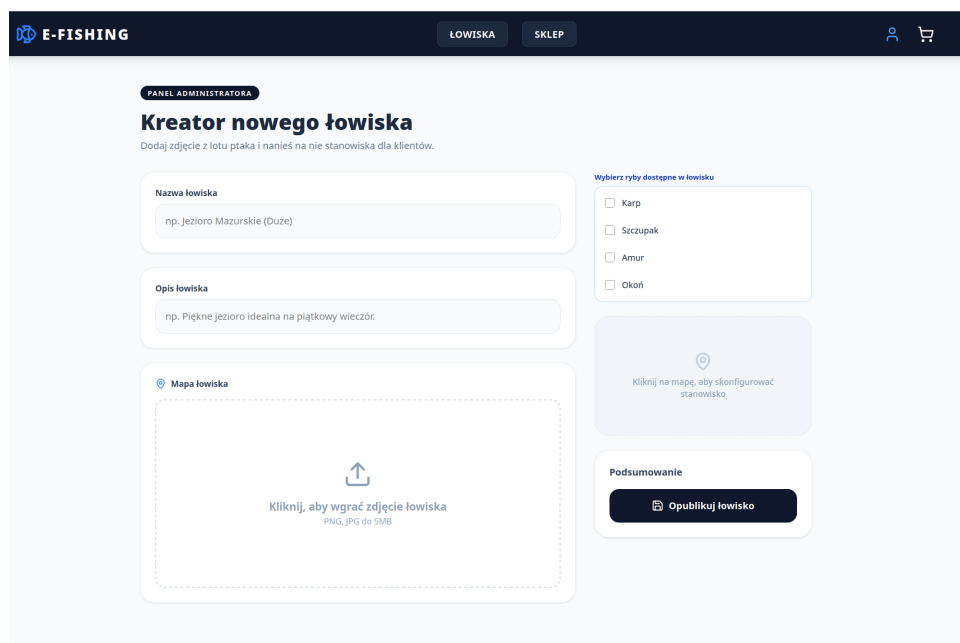
W widoku koszyka wyświetlana jest lista produktów oraz rezerwacja stanowiska. Produkty i rezerwacja można usunąć klikając ikonę kosza na śmieci. Przejście do kasy finalizuje zamówienie i pozwala na wybór metody płatności.

8.7. Wybór metody płatności



W tym kroku możliwy jest wybór płatności i finalizacja zamówienia. Koszyk jest czyszczony a zamówienie trafia do panelu klienta.

8.8. Panel administratora



Ta strona dostępna jest wyłącznie dla użytkowników zalogowanych na konto typu 'ADMIN' i jest dostępna pod adresem /admin. Ten panel umożliwia dodanie nowego jeziora wraz ze stanowiskami do serwisu. Wymagane jest wprowadzenie nazwy dla nowego jeziora. Jest również opcja dodania opisu. Poniżej pola nazwy i opisu możliwe jest wgranie obrazu z mapą danego jeziora. Po wgraniu obrazu możliwe jest dodawanie stanowisk poprzez kliknięcie lewym przyciskiem myszy. Wówczas w prawym panelu (w miejscu wypełniacza) pojawi się pole do wprowadzenia nazwy/kody stanowiska oraz przycisk zatwierdzający. Po zatwierdzeniu stanowiska będzie ono widoczne na liście w panelu po prawej. Możliwe jest usunięcie stanowiska z listy za pomocą przycisku kosza.

W panelu po prawej stronie możliwe jest również zaznaczenie gatunków ryb, które występują w danym zbiorniku. Po dodaniu wszystkich ryb oraz stanowisk możliwe jest opublikowanie łowiska. Natychmiast po pomyślnie zakończonej operacji łowisko jest dostępne w panelu rezerwacji.